BMS College of Engineering

(Autonomous College under VTU)

Department of Artificial Intelligence and Data Science

Project Work Phase-2 (23AI7PWPP2) Final Presentation on

SwarmFolio: Multi-Objective Stock Portfolio Optimization Using Particle Swarm Optimization and Genetic Algorithms

Submitted by

Vivek Boora	1BM23AD073
Ayush Pandey	1BM23AD074
Nitin	1BM24AD404

Under the guidance of

Prof. Suman M
Assistant Professor
Dept of AI & DS

Table of Contents

3 Abstract

4 Introduction

6 Literature Survey

8 Requirement Analysis and Specification

9 Design

11 Implementation

13 Testing and Validation

14 Results and Discussion

18 Conclusion

19 POs mapping matrix

Circle shows the slide number

Abstract

Three goals

- Maximise return
- Minimise volatility
- Minimise CVaR (tail loss)

under cap, floor and holding limits

Two optimisers

- Particle swarm
  MOPSO
- Genetic algorithm
  NSGA-II

One shared repair operator

What we found

0.70 vs 0.25

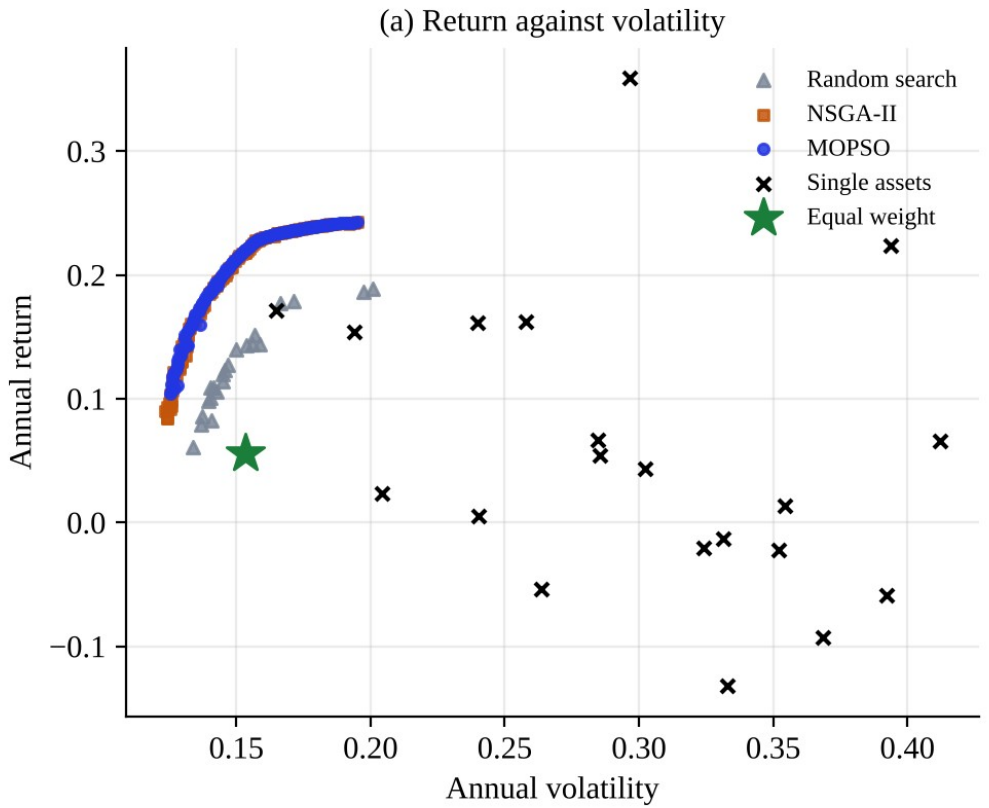
front quality, optimisers vs random search

1.34 to 0.53

Sharpe ratio, training to held-out data

A good front is not a promise of future return.

Introduction: the problem



Each dot is a feasible portfolio; the front is the best trade-off

Maximise annual return

Minimise volatility

Minimise CVaR: average loss on the worst 5% of days

Constraints applied to every portfolio

- Long-only
- Fully invested
- Cap 30% per stock
- Min holding 1%
- At most K holdings

Introduction: what we built

P

Python library

Portfolio model, repair operator, automated tests

T

Two optimisers

MOPSO and NSGA-II on one feasible space

F

Fair evaluation

Hypervolume, random search, 30 held-out markets

W

Web workbench

Watch the search and inspect any portfolio

Sustainable Development Goals

SDG 4 Quality Education

SDG 8 Decent Work and Economic Growth

SDG 9 Industry, Innovation, Infrastructure

Scope: simulated markets, long-only portfolios, no trading costs in the optimisation

Literature Survey

Paper Title	Name of Journal / Conference	Published Year	Novelty / Techniques	Limitations	Gap Identified
Portfolio selection (Markowitz)	The Journal of Finance	1952	Mean-variance model, efficient frontier	Variance only; no holding limits	Needs tail risk and constraints
Optimization of conditional value-at-risk (Rockafellar, Uryasev)	The Journal of Risk	2000	CVaR as a tail-risk measure	One risk measure at a time	Combine with return and volatility
Heuristics for cardinality constrained portfolio optimisation (Chang et al.)	Computers & Operations Research	2000	GA, tabu search, annealing with holding limits	Limits make the frontier hard to compute exactly	Fair comparison of heuristics
A fast and elitist multiobjective GA: NSGA-II (Deb et al.)	IEEE Trans. Evolutionary Computation	2002	Non-dominated sorting, crowding distance	General method, not portfolio-specific	Needs portfolio constraint handling
Handling multiple objectives with PSO (Coello et al.)	IEEE Trans. Evolutionary Computation	2004	External archive guides the swarm	General method, not portfolio-specific	Needs a swarm vs GA comparison
PSO for constrained portfolios (Cura; Zhu et al.; Deng et al.)	Nonlinear Anal. RWA; Expert Syst. Appl.	2009 to 2012	PSO with cardinality and other constraints	One scalarised objective	No three-objective front with CVaR
A portfolio model with three objectives and discrete variables (Anagnostopoulos, Mamanis)	Computers & Operations Research	2010	Tri-objective MOEAs: risk, return, number of securities	Third objective is holding count, not tail risk	Swarm, CVaR and held-out tests added here

Literature Survey: gaps and our objectives

Research gap identified	Our objective that addresses it
Held-out testing across many markets is rare	Test portfolios on 30 simulated markets with a paired test
Constraint handling differs between algorithms	One shared repair operator for the swarm and the GA
Swarm studies use a single scalarised objective	Three-objective model: return, volatility, CVaR
Few tools let you watch and inspect the search	Browser workbench with live search and portfolio inspector

Gaps refer to the papers surveyed here, not the whole field

Requirement Analysis and Specification

12

functional requirements

7

non-functional requirements

0

INR software cost, all open source

Software stack

Python 3.13 | NumPy | pandas | Matplotlib | SciPy | pytest | Browser canvas

Default run: 100 individuals, 150 iterations, 20 assets finishes in about 1.5 seconds

Design: architecture



Design: methodology

One repair operator, shared by both algorithms

Clip negatives to zero → Keep the K largest → Drop tiny weights → Add assets if the cap needs more → Cap at 30% and redistribute → Normalise to sum 1

Six experiments

E1 — Front quality
10 seeds against random search

E2 — Holding limits
K = none, 10, 8 and 5

E3 — Number of assets
10 to 60 assets

E4 — Search budget
5 to 200 iterations

E5 — Held-out test
30 simulated markets

E6 — Operator ablation
with and without

Seeded factor-model markets with fat tails and a crisis period; 70% training, 30% held-out

Implementation: the two optimisers

MOPSO (particle swarm)

Particles start as feasible portfolios

↓

Leader chosen from the archive by crowding

↓

Velocity update, then repair

↓

Archive keeps the non-dominated set

100 particles | inertia 0.9 to 0.4 | c1 = c2 = 1.5

NSGA-II (genetic algorithm)

Population of feasible portfolios

↓

Tournament by rank and crowding

↓

SBX crossover, mutation and swap

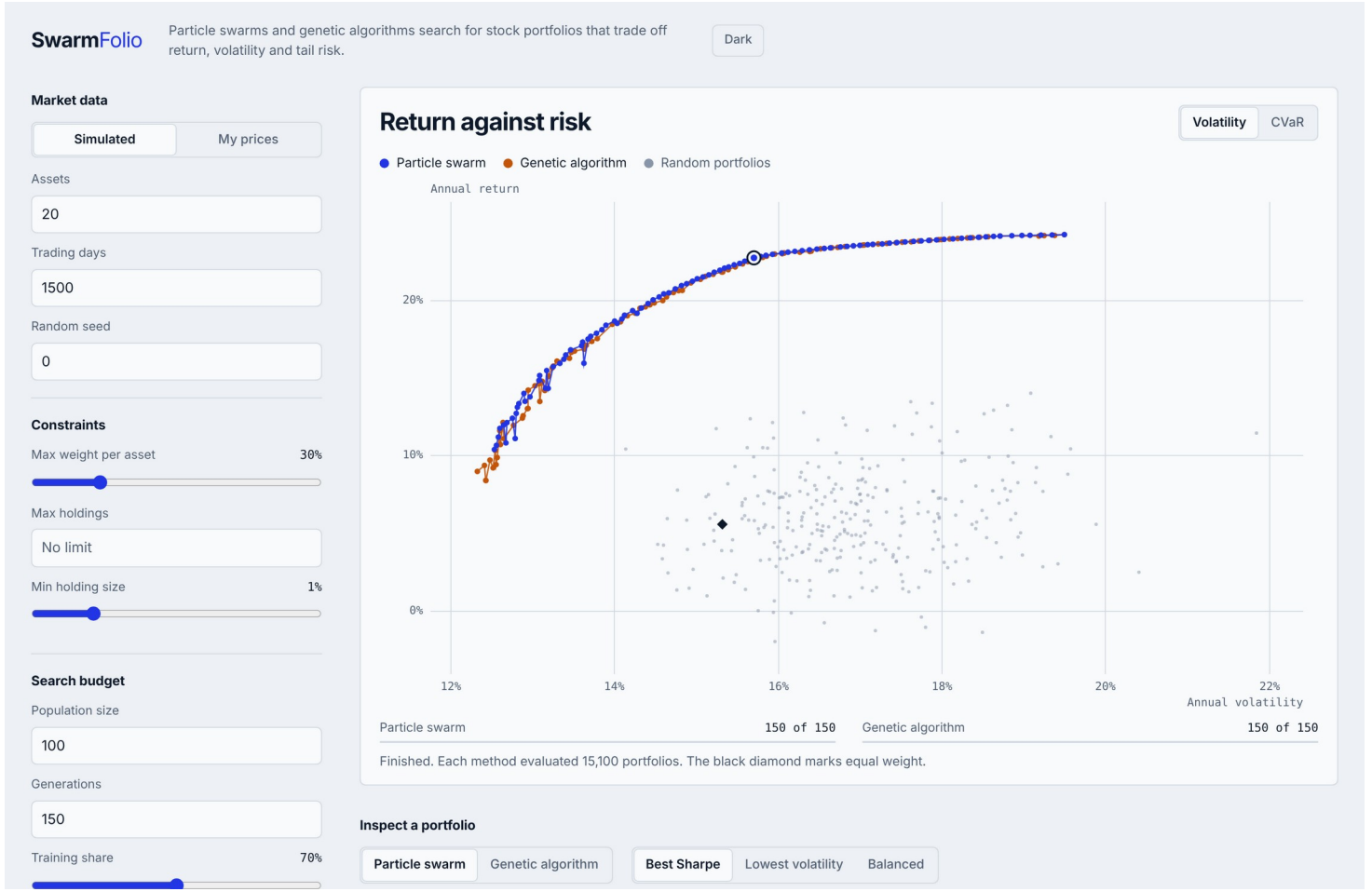
↓

Parents and children: keep the best

100 individuals | SBX index 15 | swap mutation 0.2

Same budget for both: 15,100 objective evaluations per run

Implementation: the workbench



Watch the search

Both populations converge on their fronts, live

Inspect any portfolio

Click a point: weights, training and held-out results

Bring your own prices

Upload a CSV, set constraints, press Run

Testing and Validation

14

automated tests pass

19

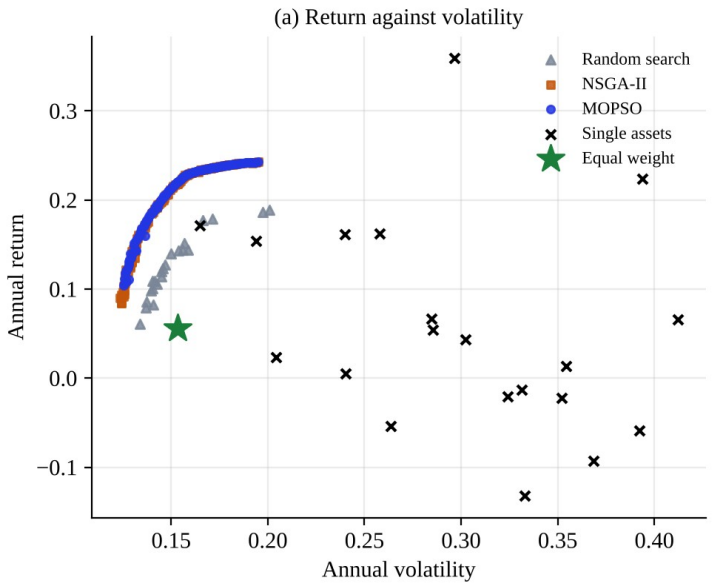
system test cases: server and browser

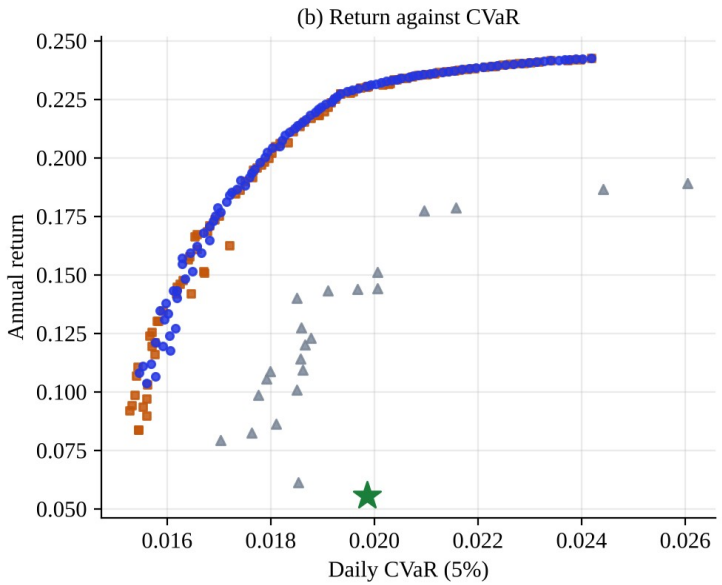
0

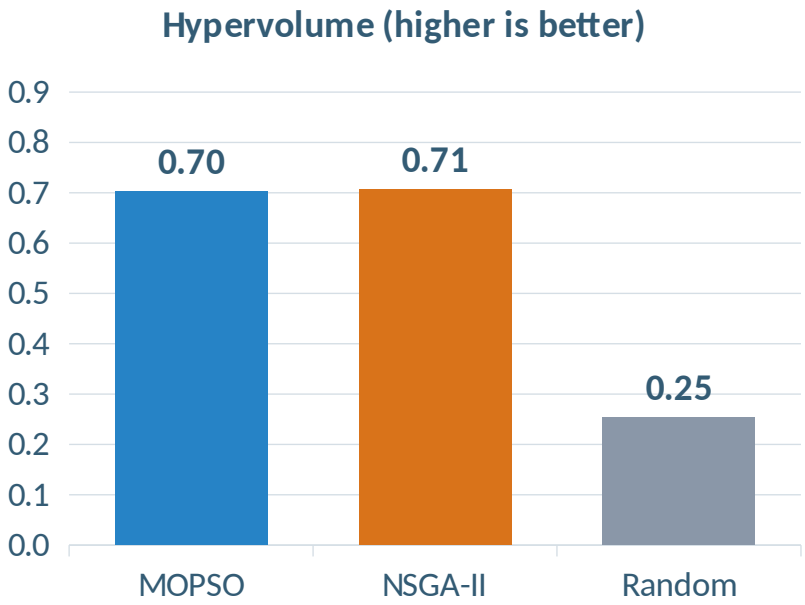
constraint violations in every experiment run

Component	Algorithm	System
Repair, objectives, Pareto maths	Feasible, non-dominated, reproducible, beats random	Server jobs, error paths, browser, phone, dark mode

Results: front quality







100 vs 20

non-dominated portfolios per run: optimisers vs random

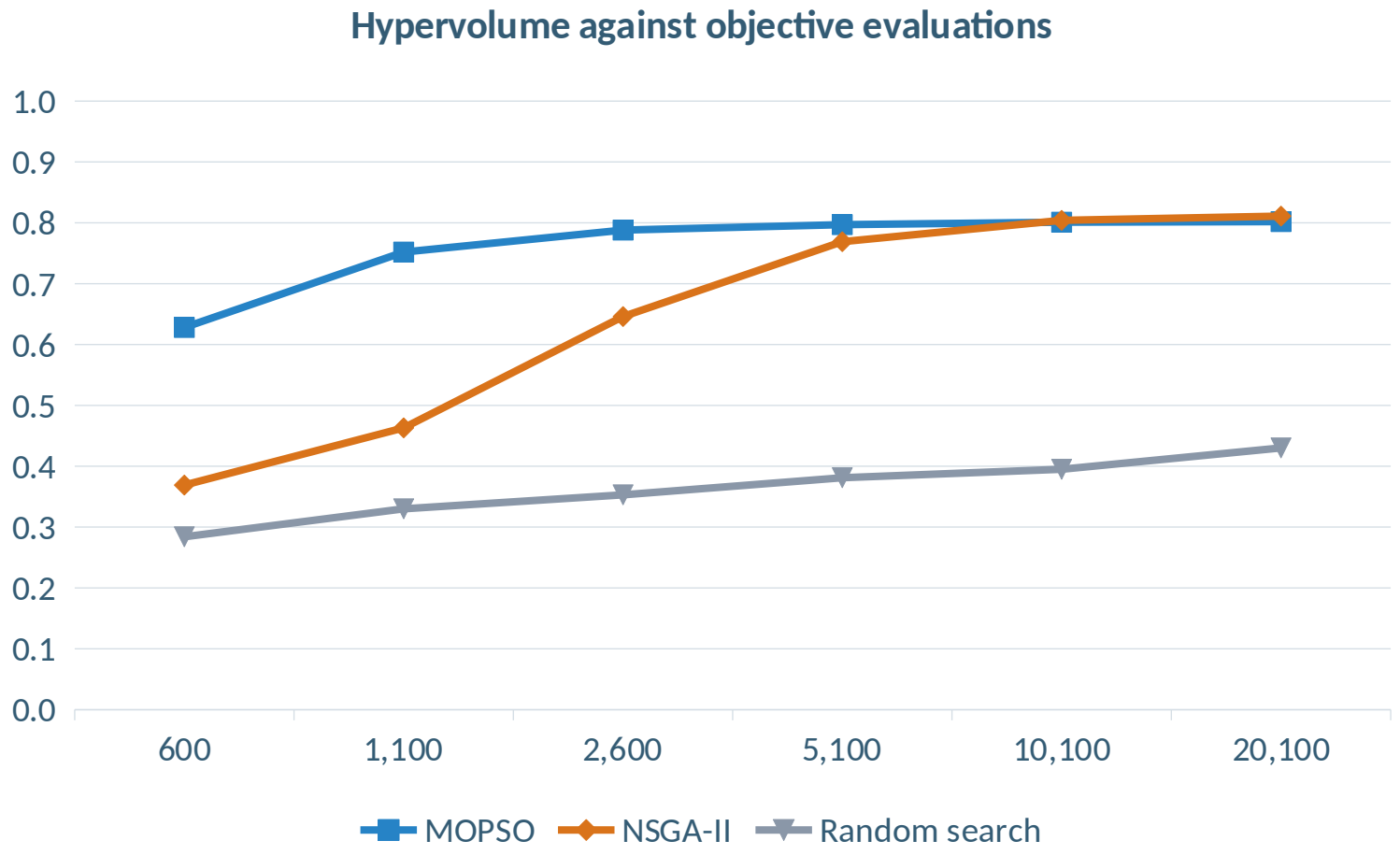
p = 0.002

both optimisers vs random, paired test over 10 seeds

0.70 vs 0.71

swarm vs genetic: a statistical tie in practice

Results: search budget



Up to 50 iterations

Swarm ahead in 5 of 5 seeds

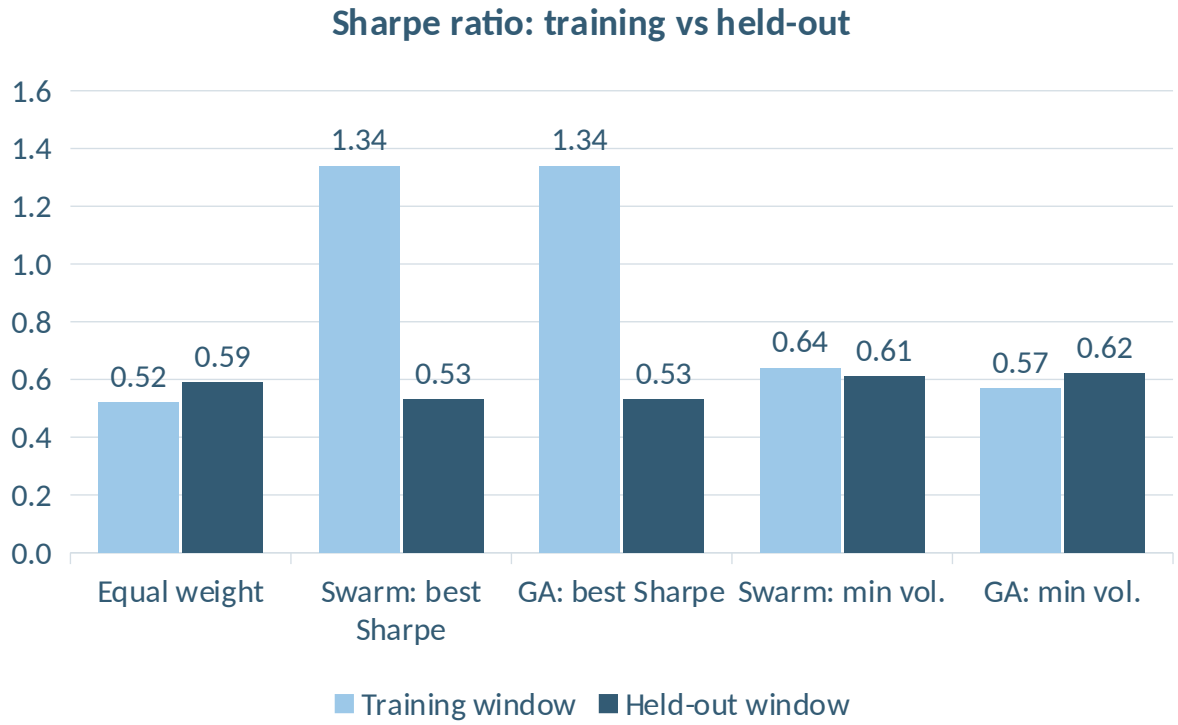
From 100 iterations

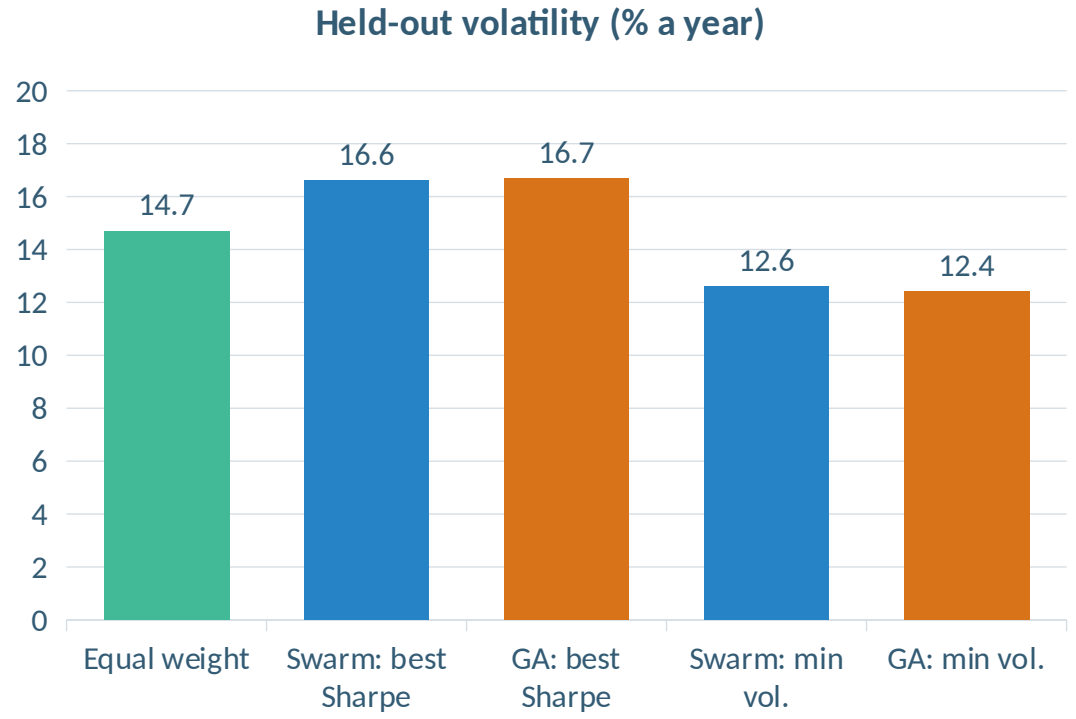
Genetic algorithm ahead in 5 of 5 seeds

Random search

Stays far behind at every budget

Results: held-out performance

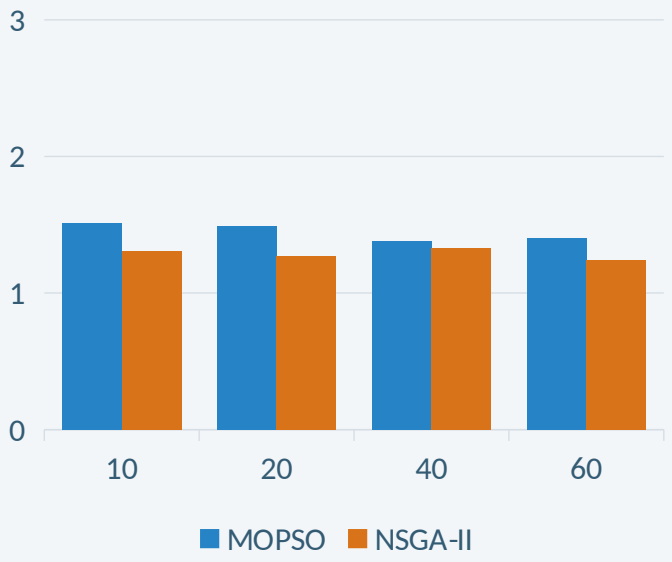




Return does not transfer to new data. Risk does: lower held-out volatility in 30 of 30 markets

Results: robustness checks

Scale



Seconds per run, 10 to 60 assets
about 1.5 s whatever the size

Constraints

0

violations of the cap or the holding limit, for limits of none, 10, 8 and 5 holdings

Operator ablation

No gain

No turbulence: p = 0.64

No swap mutation: p = 0.55

Neither changes front quality at these settings

Conclusion

1

Both optimisers work

Far better fronts than random search; every constraint respected

2

Return does not transfer

Best-Sharpe picks: 1.34 in training, 0.53 held out; equal weight 0.59

3

Risk does

Minimum-volatility picks were calmer than equal weight in 30 of 30 markets

Future scope

- Real market data
- Shrinkage estimators
- Trading costs as an objective
- Hosted classroom version

Program Outcomes Attained

3 High | 2 Medium | 1 Low

PO1 Engineering knowledge **Level 3**	**PO2** Problem analysis **Level 3**	**PO3** Design of solutions **Level 2**	**PO4** Investigation of problems **Level 3**	**PO5** Modern tool usage **Level 3**	**PO6** Engineer and the world **Level 1**	**PO7** Ethics **Level 2**
PO8 Teamwork **Level 3**	**PO9** Communication **Level 2**	**PO10** Project management **Level 2**	**PO11** Life-long learning **Level 2**	**PSO1** Design AI systems **Level 3**	**PSO2** Decisions with AI and data **Level 2**	**PSO3** End-to-end systems **Level 3**

Each level is justified in the project report (attainment sheet).